SYSTEM_ARCHITECTURE.md

Version: 1.0

Status: Approved

Architecture Style: Modular Monolith with Clean Architecture

---

1. Architecture Goal

CommercePilot shall be built as a production-ready, modular, AI-powered SaaS platform that integrates with existing e-commerce systems while remaining easy to maintain, test, and scale.

The architecture prioritizes:

- Maintainability
- Scalability
- Security
- Modularity
- Multi-tenancy
- AI integration
- Event-driven workflows

---

2. Architectural Principles

- Clean Architecture
- Domain-Driven Design (DDD)
- SOLID Principles
- API-First Design
- Event-Driven Processing
- Human-in-the-Loop AI
- Multi-Tenant Isolation
- Security by Design

---

3. High-Level Components

Frontend

Technology:

- Next.js
- TypeScript

- Tailwind CSS
- shadcn/ui

Responsibilities:

- Admin Dashboard
- Authentication
- Order Review
- Analytics
- Settings
- User Management

Backend

Technology:

- NestJS
- TypeScript

Responsibilities:

- Business Logic
- REST APIs
- Authentication
- AI Processing
- Integrations
- Background Jobs

Database

Technology:

- PostgreSQL
- Prisma ORM

Responsibilities:

- Tenant Data
- Orders
- Messages
- Audit Logs
- Users
- Product Catalog

Queue System

Technology:

- Redis
- BullMQ

Responsibilities:

- AI Processing
- Email Sending
- Retry Jobs
- Scheduled Tasks

AI Layer

Technology:

- Gemini Flash (MVP)

Responsibilities:

- Intent Detection
- Entity Extraction
- Product Matching
- Confidence Scoring
- Order Summarization

---

4. Backend Module Structure

Each module owns its own business logic.

Modules:

- Auth
- Tenants
- Users
- Products
- Orders
- Customers
- WhatsApp
- AI Engine
- Integrations
- Dashboard
- Notifications
- Audit Logs
- Settings

Modules communicate through services and domain events.

---

5. Layered Architecture

Presentation Layer

↓

Application Layer

↓

Domain Layer

↓

Infrastructure Layer

Dependencies flow inward only.

Infrastructure never contains business rules.

---

6. AI Processing Pipeline

Customer Message

↓

Webhook Receiver

↓

Queue Job

↓

Intent Detection

↓

Product Retrieval (RAG)

↓

Entity Extraction

↓

Confidence Calculation

↓

Draft Order Generation

↓

Owner Approval

↓

WooCommerce Synchronization

↓

Customer Notification

---

7. Integration Layer

Supported Integrations (MVP)

- WhatsApp Cloud API
- WooCommerce REST API
- Email Service
- Supabase Storage

Future Integrations

- Shopify
- Facebook Messenger
- Instagram
- Courier APIs
- Payment Gateways

All integrations are isolated behind adapter interfaces.

---

8. Multi-Tenant Architecture

Every request contains a Tenant ID.

Each tenant has isolated:

- Users

- Products
- Customers
- Orders
- Messages
- Analytics
- AI Context

No cross-tenant access is permitted.

---

9. Security Architecture

Authentication:

JWT

Authorization:

Role-Based Access Control (RBAC)

Roles:

- Super Admin
- Tenant Owner
- Staff

Sensitive data must be encrypted where appropriate.

Audit logs are immutable.

---

10. Event-Driven Workflow

Business events include:

- MessageReceived
- AIProcessingStarted
- AIProcessingCompleted
- DraftOrderCreated
- OrderApproved
- OrderRejected
- OrderSynced
- EmailSent
- NotificationSent

These events trigger background jobs without blocking user requests.

---

11. Error Handling

Every module shall:

- Validate input
- Return meaningful errors
- Log failures
- Retry transient failures
- Prevent duplicate processing

External API failures must use retry policies.

---

12. Logging & Monitoring

Structured logging is required.

Log categories:

- API
- AI
- Authentication
- Database
- Integrations
- Queue Jobs
- Security Events

Every request receives a correlation ID for tracing.

---

13. Scalability Strategy

Current:

Single NestJS application (Modular Monolith)

Future:

Individual modules may be extracted into microservices if scaling requires it.

The modular boundaries established now should make this migration possible without major redesign.

---

14. Deployment Architecture

Frontend

↓

Vercel

Backend

↓

Render

Database

↓

Supabase PostgreSQL

Queue

↓

Redis

AI

↓

Gemini API

Email

↓

Resend

---

15. Architecture Rules

- Business logic must never exist in controllers.
- Controllers only orchestrate requests and responses.
- Services contain application logic.
- Domain models contain business rules.
- Infrastructure contains external integrations.
- Every external dependency must be abstracted behind an interface.
- No module may directly access another module's database tables.
- Shared functionality belongs in common libraries, not duplicated across modules.

---

16. Architecture Decision

CommercePilot will use a Modular Monolith architecture for the MVP.

This provides production-grade maintainability while reducing operational complexity.

The architecture intentionally supports future migration to microservices if business growth requires independent scaling.